Il costrutto if e la scelta condizionale

Lezione 8 — 1IOT, SST-ICT-01 — Programmi che decidono cosa fare: if, if-else, annidamento

29/09/2026

1 Ripasso: printf e scanf

Nota

Ripasso veloce della lezione precedente. **`printf`** produce quello che si chiama **output**: nel nostro caso, l'**output** finisce sul monitor. **`scanf`** permette di ottenere un **input**: la nostra periferica di **input** è la tastiera. Se immaginiamo il programma come una **scatola**, siamo in grado sia di far produrre alla scatola qualcosa che esce e viene mostrato all'utente, sia di inserire informazioni dentro la scatola.

Attenzione

Questo concetto (scatola che riceve **input** ed emette output) è importantissimo perché tornerà in continuazione quando si userà **Arduino**: Arduino è una scheda alla quale possiamo sia inviare informazioni sia farci restituire informazioni, a seconda di cosa stiamo facendo con essa.

Definizione

Riepilogo della sintassi:

```
// stampa; %d per interi, %f per float
printf("testo %d", variabile);
// legge da tastiera; MAI dimenticare &
scanf("%d", &variabile);
```

`printf`, una volta finita la stampa, non va **mai** a capo da solo: serve `\n` se si vuole andare a capo esplicitamente. `scanf` accetta esclusivamente numeri (**mai** testo libero, non è un'intelligenza artificiale conversazionale): `%d` per un **intero**, `%f` per un numero **con la virgola**.

Esempio

Esercizio dell'**area** del quadrato, già svolto e ripreso come base di partenza per la lezione:

```
int L;
printf("Inserisci il lato del quadrato: ");
scanf("%d", &L);

int area;
area = L * L;

printf("L'area del quadrato vale %d", area);
```

2 Come esegue la macchina: una sequenza di operazioni

Definizione

Il codice dell'esempio precedente viene eseguito dalla macchina come una **sequenza ordinata di operazioni**, sempre nello stesso ordine:

1. Crea la **variabile** `L` (`int L;`)
2. Stampa il messaggio (**`printf`**)
3. Acquisisce il dato da **tastiera** (`scanf`)
4. Crea la **variabile** `area` (`int area;`)
5. Calcola `L * L`
6. Stampa il risultato (**`printf`**)

Un programma è infatti definito come una **lista ordinata di operazioni**: eseguito 42 volte, farà sempre e solo questa identica sequenza, **mai** qualcosa di diverso.

Attenzione

Il problema: un programma che fa sempre e solo la stessa identica cosa non è utile. Analogia usata in aula: un videogioco in cui l'unica azione possibile è sempre la stessa ("premi qui per iniziare", fine del gioco) non sarebbe divertente né utile. Serve un programma in grado di **scegliere cosa fare** in base alla situazione.

3 Il diagramma di flusso: sequenza vs. scelta

Nota

Rappresentazione grafica usata in aula: un **rettangolo** rappresenta un'azione da compiere (es. creare una **variabile**, stampare un messaggio). Una **sequenza** di rettangoli collegati da frecce rappresenta il flusso lineare di operazioni, come i livelli di un videogioco: primo, secondo, terzo… sempre nello stesso ordine.

Definizione

Per rappresentare una **scelta** nel flusso serve un nuovo simbolo, il **rombo**: al suo interno si scrive una domanda con risposta **vera**/falsa (sì/no). Dal **rombo** partono due frecce, una per ciascuna risposta possibile, che portano a percorsi diversi del programma.

Esempio

Analogia della vita quotidiana usata per introdurre il concetto: portare sempre l'ombrello, pioggia o sole, non è sbagliato, ma è scomodo. Meglio: "mi sveglio, mi lavo, mi vesto, faccio colazione (sempre, comunque). Guardo fuori: se piove, prendo l'ombrello; se non piove, lo lascio a casa." Questo si chiama **costrutto decisionale**: permette di fare **scelte**, cioè di far

seguire al flusso un percorso o l'altro in base a una **condizione**.

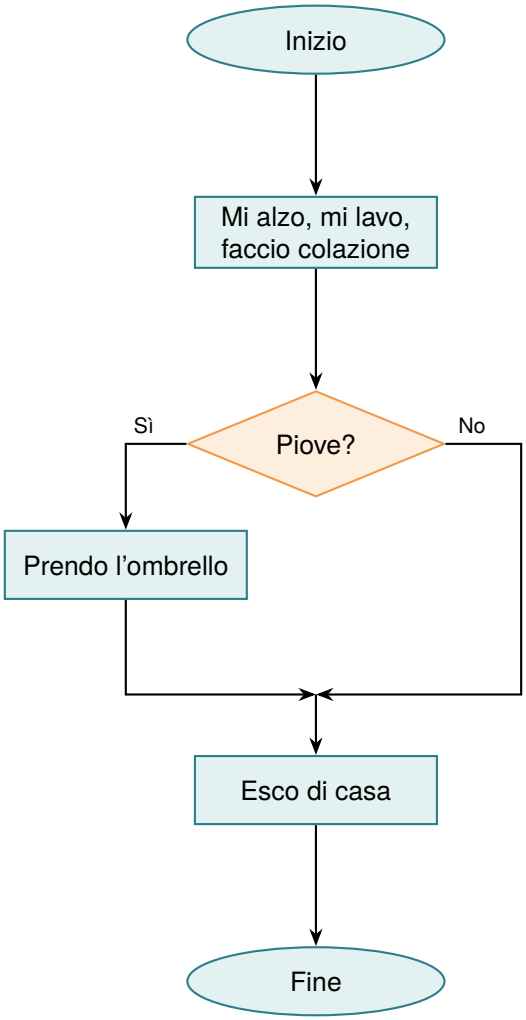


Figura 1: Diagramma di flusso disegnato in aula: il rombo "Piove?" smista il flusso su due percorsi diversi, che si ricongiungono prima di proseguire.

4 Il costrutto if

Definizione

`if` (in inglese "se") è il **costrutto** che traduce in codice il rombo del **diagramma di flusso**:

```
if (condizione)
{
    // codice eseguito SOLO se la condizione e' vera
}
```

Le **due graffe** sono obbligatorie: creano un contenitore che delimita esattamente quali istruzioni fanno parte del **blocco** "da eseguire se **vero**" e quali sono invece il normale proseguimento del codice. Senza le **graffe** la macchina non saprebbe distinguere le due cose.

Nota

`if` non è una semplice **funzione** della libreria C: è un **costrutto del linguaggio**, cioè una parola chiave presente in praticamente ogni linguaggio di programmazione testuale esistente (Python, C#, Delphi/Object Pascal...): fare **scelte** condizionali è un'esigenza così fondamentale che mancare del tutto sarebbe un'eccezione rarissima. Per i PLC il docente ha anticipato che, quando li si affronterà, si troverà comunque un modo di esprimere **condizioni** simili: nel linguaggio a contatti (**ladder logic**), il più tradizionale per i PLC, non esiste però una parola chiave `if` **vera** e propria — la stessa logica condizionale si ottiene componendo contatti e bobine sullo schema a scala; linguaggi PLC più recenti come lo **Structured Text** hanno invece un **`if`** testuale del tutto simile a quello del C.

Definizione

La **condizione** tra parentesi si costruisce con gli **operatori condizionali** già visti nelle lezioni precedenti (es. `>`, `>=`, `<`, **`==`**): espressioni che valgono sempre o vero o falso, **mai** una via di mezzo.

5 Esempio guidato: il voto di un allievo

Esempio

Esempio sviluppato dal vivo in aula, con esecuzione reale su un compilatore online: un programma che chiede il voto (in decimi, valore **intero**) di un allievo. Prima versione scritta in diretta, in cui il messaggio positivo viene stampato sempre, e solo in caso di voto basso si aggiunge un messaggio in più con l'**`if`**:

```c
int voto;
printf("Inserisci il voto dell'allievo: ");
scanf("%d", &voto);

printf("Bravo!");

if (voto < 6)
{
    printf("Sei bocciato!");
}
```

Attenzione

Il bug scoperto insieme in aula: con voto `8` il programma stampa correttamente solo "Bravo!". Ma con voto `3` stampa **"Bravo!Sei bocciato!"**, cioè **entrambi** i messaggi: il primo `printf("Bravo!")` viene eseguito comunque, perché sta fuori dall'`if` ed è quindi una normale istruzione in **sequenza**, non collegata ad alcuna **condizione**. L'`if` da solo può solo **aggiungere** un'istruzione quando la **condizione** è vera: non può **sostituire** o **impedire** un'istruzione già scritta fuori da esso. Manca un modo per dire esplicitamente cosa fare *altrimenti* (cioè, invece di stampare "Bravo!").

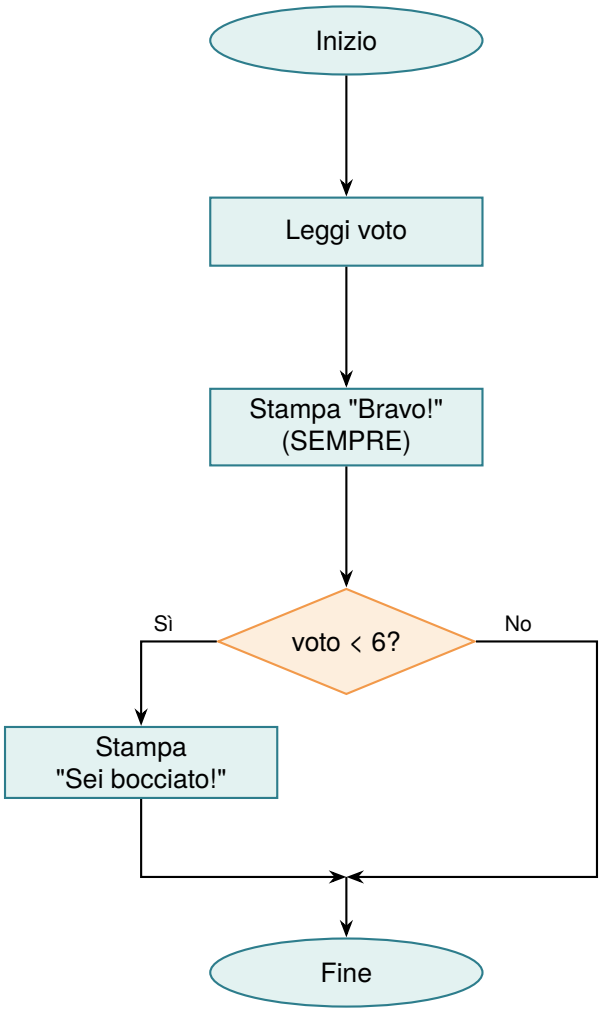


Figura 2: Diagramma di flusso della prima versione (con il bug): "Stampa Bravo!" viene eseguita SEMPRE, indipendentemente dal ramo dell'if, perché sta fuori dal blocco condizionale.

6 La variante if-else

Definizione

`if...else` aggiunge il **ramo** "altrimenti": se la condizione è **vera** si esegue il primo blocco, se è **falsa** si esegue il **blocco** `else`. I due blocchi si escludono sempre a vicenda: viene eseguito uno solo dei due.

```c
int voto;
printf("Inserisci il voto dell'allievo: ");
scanf("%d", &voto);

if (voto < 6)
{
    printf("Sei bocciato!");
}
else
{
    printf("Bravo!");
}
```

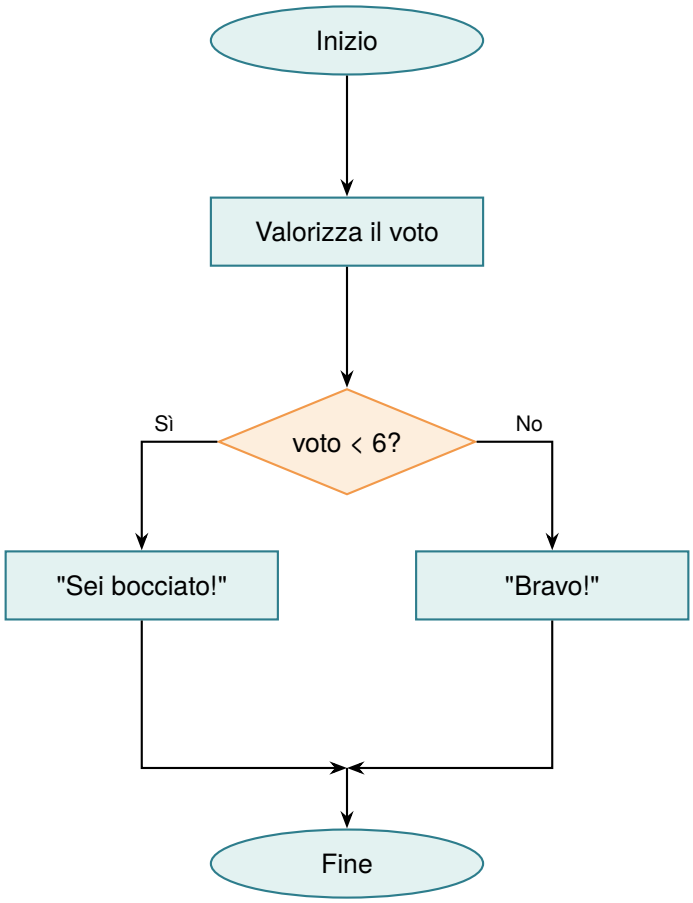


Figura 3: Diagramma di flusso dell'`if-else`: i due rami si escludono a vicenda (viene sempre eseguito uno solo dei due blocchi), poi il flusso si ricongiunge prima di proseguire.

Esempio

Test eseguiti dal vivo in aula sul programma sopra: con voto `8` il programma stampa "Bravo!"; con voto `3` il programma stampa "Sei bocciato!". Il comportamento del programma cambia effettivamente in base al dato inserito dall'utente: non è più una **sequenza** fissa.

Nota

Lo stesso problema si può scrivere "al contrario", **invertendo** condizione e messaggi, con risultato identico:

```c
if (voto >= 6)
{
    printf("Bravo!");
}
else
{
    // bocciato
    printf("Ti devi impegnare di piu'!");
}
```

Per acquisire un voto **con la virgola** basta cambiare lo **specificatore di formato** in `scanf("%f", &voto);`, dichiarando `voto` come `float`: il resto della logica dell'**`if-else`** non cambia.

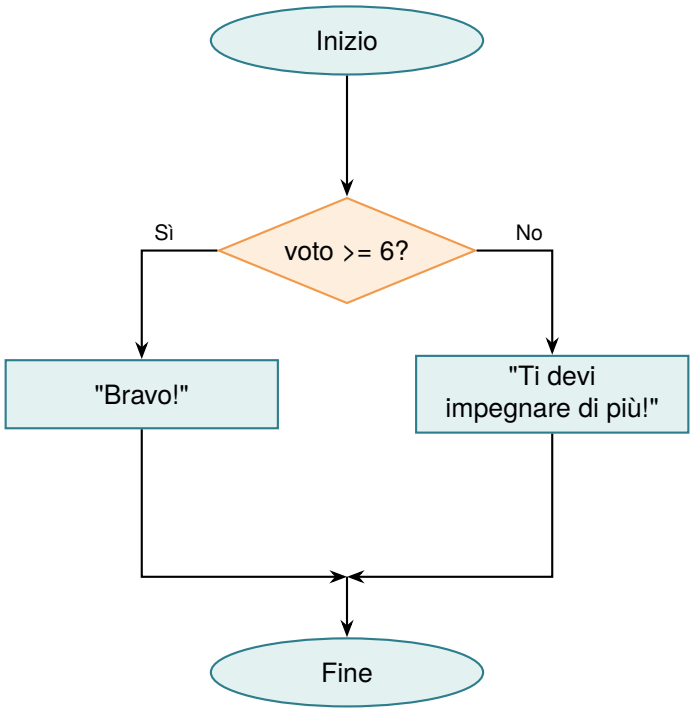


Figura 4: Diagramma di flusso della versione "al contrario": stessa logica dell'if-else precedente, condizione e messaggi invertiti, risultato identico.

7 Attenzione ai confini: maggiore vs. maggiore o uguale

Attenzione

Errore molto comune: scrivere `if (voto > 6)` quando in realtà si vuole includere anche il 6 come voto sufficiente. La macchina è **rigida**, non interpreta le intenzioni: se `voto` vale esattamente `6`, la **condizione** `voto > 6` è **falsa** (6 non è maggiore di 6), quindi con un **`if`** scritto così un allievo con 6 risulterebbe "bocciato". Per includere anche il valore limite serve l'operatore **maggiore o uguale**: `voto >= 6`. L'**errore** in questi casi è sempre del programmatore, non della macchina: la **condizione** va scritta con precisione.

8 L'annidamento: più di due possibilità

Definizione

Quando le possibilità da distinguere sono più di due (non solo "bocciato/promosso" ma, ad esempio, quattro fasce di giudizio), si usa l'**annidamento**: un **`if`** dentro un altro **`if`**, quanti livelli servono. Ogni `if` **annidato** mantiene comunque le proprie due **graffe** obbligatorie.

Esempio

Obiettivo discusso in aula: costruire una stratificazione di giudizi sul voto — sotto il 5 "bocciato", tra il 5 e il 6 "devi impegnarti di più", tra il 6 e il 7 "bravo", sopra il 7 "genio". Ogni fascia richiede una **condizione** dedicata, costruita con attenzione a come le fasce si delimitano a vicenda.

Attenzione

Errore classico da evitare: scrivere due `if` **consecutivi** invece che **annidati**, quando le **condizioni** non si escludono a vicenda. Esempio discusso in aula:

```c
// NON annidato: if separato
if (voto >= 5.7)
{
    printf("Bene");
}
// NON annidato: if separato
if (voto > 7)
{
    printf("Bravo");
}
```

Con voto `8`, questo codice stampa **sia** "Bene" **sia** "Bravo", perché 8 soddisfa entrambe le condizioni indipendenti: non è il comportamento voluto, se si vuole un solo messaggio per voto.

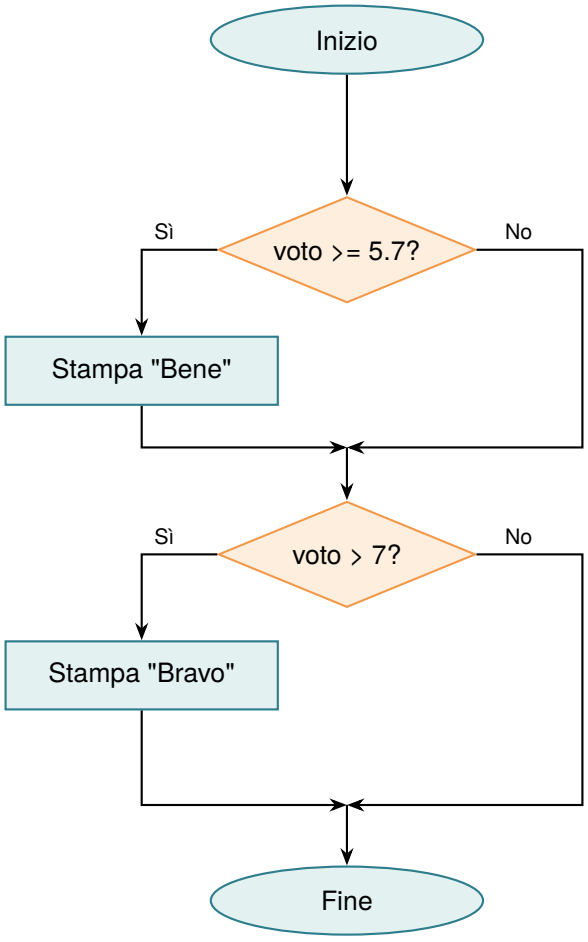


Figura 5: Diagramma di flusso dell'errore: due `if` CONSECUTIVI (non annidati) vengono entrambi valutati sempre, indipendentemente dal risultato del primo – con voto 8 si stampano sia "Bene" sia "Bravo".

Definizione

Soluzione corretta con annidamento: il secondo `if` va scritto dentro il blocco del primo, così da essere valutato **solo se** si è già certi che la prima condizione è **vera**:

```c
if (voto >= 5.7)
{
    if (voto > 7)
    {
        printf("Ottimo");
    }
    else
    {
        printf("Sufficiente");
    }
}
```

Con questa struttura: se `voto` vale `5.7`, `6`, o `6.9` il programma stampa "Sufficiente" (è entrato nel primo **if** ma non nel secondo); se `voto` vale `7.1` stampa "Ottimo" (soddisfa entrambe le **condizioni**); se `voto` è sotto `5.7` non entra proprio nel **blocco** (in questo esempio semplificato, nessun messaggio viene stampato per quel caso, che andrebbe gestito con un **else** esterno).

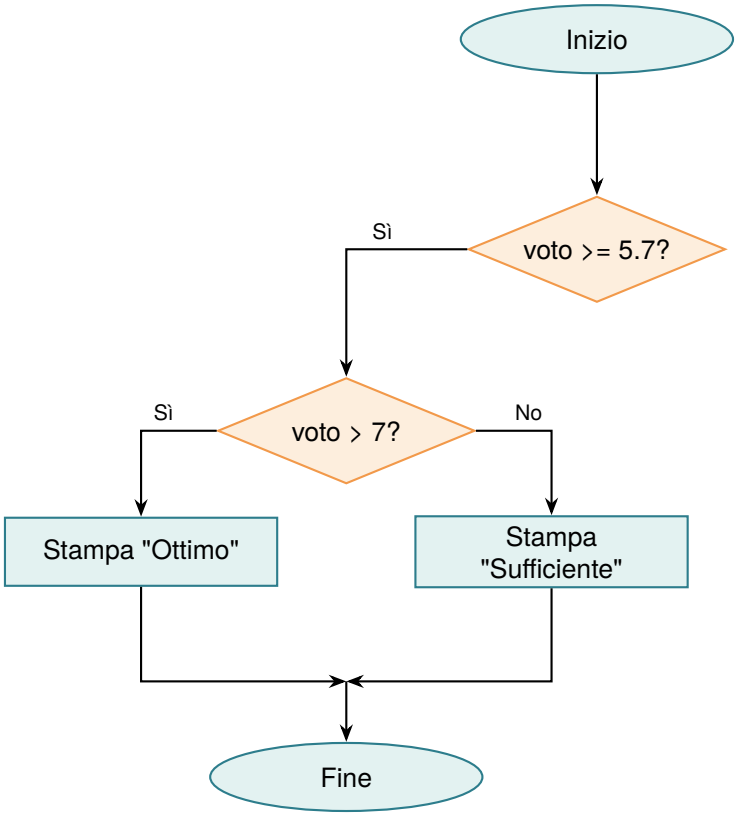


Figura 6: Diagramma di flusso della soluzione corretta con annidamento: il secondo `if` (voto > 7?) viene valutato SOLO se il primo è già vero – così non si stampano mai due messaggi insieme. Il ramo "No" del primo if (voto sotto 5.7) non è rappresentato: in questo esempio semplificato nessun messaggio viene stampato per quel caso.

9 Cosa ricordare di questa lezione

- Un programma senza **scelte** esegue sempre la stessa identica **sequenza** di operazioni, in ogni esecuzione: non è utile.
- Il **diagramma di flusso** rappresenta le scelte con un **rombo** (domanda vero/falso) e due frecce di uscita.
- `if (condizione) { ...}` esegue il blocco **solo se** la condizione è vera; le **graffe** sono sempre obbligatorie.
- `if ...else ...` aggiunge il **ramo** "altrimenti": i due blocchi si escludono sempre a vicenda.
- Attenzione ai **confini**: `>` esclude il valore limite, **`>=`** lo include. La macchina applica la **condizione** esattamente come scritta, senza interpretare le intenzioni.
- Per più di due possibilità si usa l'**annidamento** (`if` dentro `if`): due `if` consecutivi non **annidati**, con condizioni che si sovrappongono, possono eseguire più blocchi contemporaneamente per **errore**.
- L'**annidamento** va limitato per non compromettere la leggibilità del codice.

Esempio

Da fare per la prossima lezione: esercitarsi con l'**`if`** e l'`if-else` in aula, costruendo **condizioni** su esempi numerici (voti, misure), prima di affrontare l'**annidamento** su più livelli.